

FCM I Programming Assignment 2

Charles Ohanian

October 16, 2024

1 Executive Summary

In this report, we consider the development and testing of an algorithm that computes the LU factorization of a full-rank matrix $A \in \mathbb{R}^{n \times n}$. The algorithm is split into two cases: no pivoting and partial pivoting. Complete pivoting is briefly discussed, but a routine for this technique is not included in this report. Both the no pivoting and partial pivoting routines use the immediate update form of algorithm based on rank-one updates. Routines for solving the matrix equation $Ax = b$ with a factorized matrix A are also developed and tested. The routines are tested with matrices of special structure and with large quantities of random nonsingular matrices. The error is found to be very minimal for both factorization techniques tested and for the solver, with the only error coming from machine and roundoff error.

2 Statement of the Problem

LU factorization is the process of breaking down a square matrix A into the product of two matrices: a unit lower triangular matrix L and an upper triangular matrix U . For reasons discussed in the next section, LU factorization can be quite useful when attempting to solve the matrix equation $Ax = b$. When factoring A into L and U , it is not necessary to store these full matrices since the entries above or below the diagonal, respectively, are all zeros. Thus, the nonsingular matrix A that is passed through the factorization routine can be updated so that the lower triangular part contains the entries from L and the upper triangular part, including the diagonal, contains the entries from U . Additionally, in the case of partial pivoting, the entire permutation matrix does not need to be stored, as it is mostly zeros. Instead, this information can be stored in a 1D-array that tracks the order of the rows. The factorization must be efficient and numerically stable, with special attention to scenarios where candidate pivot sets are too small or contain zero values. It is also important to include empirical evaluation against randomly generated matrices and structured test cases to assess the performance and accuracy of the method across a range of problem sizes and pivoting strategies.

3 Description of the Mathematics

LU factorization is a method of decomposing a nonsingular square matrix $A \in \mathbb{R}^{n \times n}$ into a product of two matrices, $L \in \mathbb{R}^{n \times n}$ and $U \in \mathbb{R}^{n \times n}$, such that $A = LU$. The matrix L is a

unit lower triangular matrix, meaning all of its diagonal elements are 1 and its only nonzero elements are below the diagonal. The matrix U is an upper triangular matrix, where all the elements below the diagonal are 0. LU factorization is very useful in solving systems of linear equations. Once we have the LU decomposition, solving $Ax = b$ becomes easier, as it can be broken down into solving two triangular systems:

1. Solve $Ly = b$ using forward substitution (since L is lower triangular)
2. Solve $Ux = y$ using backward substitution (since U is upper triangular)

LU factorization can be done in three main ways. With no pivoting, no rows are swapped and the A is iterated through as is. With partial pivoting, rows of A are interchanged to improve numerical stability. With complete pivoting, both rows and columns of A are interchanged to further improve numerical stability. This report will delve into the mathematics behind the no pivoting and partial pivoting methods.

3.1 No Pivoting

LU factorization with no pivoting is intimately tied to Gaussian elimination. Gaussian elimination operates by transforming the matrix A into an upper triangular matrix U through a series of row operations. These row operations can be represented as the multiplication of A by elementary matrices, denoted as $M_1^{-1}, M_2^{-1}, \dots, M_{n-1}^{-1}$. LU factorization starts with a dense nonsingular matrix $A \in \mathbb{R}^{n \times n}$:

$$A = \begin{bmatrix} \alpha_{11} & \alpha_{12} & \dots & \alpha_{1n} \\ \alpha_{21} & \alpha_{22} & \dots & \alpha_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ \alpha_{n1} & \alpha_{n2} & \dots & \alpha_{nn} \end{bmatrix}$$

The first step in Gaussian elimination is to eliminate the entries below the pivot element α_{11} in the first column. This is done by subtracting multiples of the first row from each subsequent row. For $j \in \{2, 3, \dots, n\}$, the multiplier is:

$$\lambda_{j1} = \frac{\alpha_{j1}}{\alpha_{11}}, \text{ where } \alpha_{11} \neq 0.$$

Subtracting λ_{j1} times the first row from row j , results in the transformation $A^{(1)} = M_1^{-1}A$, where:

$$M_1^{-1} = \begin{bmatrix} 1 & 0 & 0 & \dots & 0 \\ -\lambda_{21} & 1 & 0 & \dots & 0 \\ -\lambda_{31} & 0 & 1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ -\lambda_{n1} & 0 & 0 & \dots & 1 \end{bmatrix}$$

The resulting matrix $A^{(1)}$ has the form:

$$A^{(1)} = \begin{bmatrix} \alpha_{11} & \alpha_{12} & \alpha_{13} & \dots & \alpha_{1n} \\ 0 & \tilde{\alpha}_{22} & \tilde{\alpha}_{23} & \dots & \tilde{\alpha}_{2n} \\ 0 & \tilde{\alpha}_{32} & \tilde{\alpha}_{33} & \dots & \tilde{\alpha}_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \tilde{\alpha}_{n2} & \tilde{\alpha}_{n3} & \dots & \tilde{\alpha}_{nn} \end{bmatrix}$$

It can be observed that the entries in the first column below the diagonal have been zeroed out, or eliminated. Continuing this process for subsequent columns, we generate the matrices $M_2^{-1}, M_3^{-1}, \dots, M_{n-1}^{-1}$, each of which eliminates the appropriate elements to create zeros below the diagonal. By multiplying all of these elementary matrices together, we obtain:

$$M_{n-1}^{-1} \cdots M_2^{-1} M_1^{-1} A = U,$$

where U is an upper triangular matrix. Rearranging, we see that $A = M_1 M_2 \cdots M_{n-1} U$, where M_i is an elementary lower triangular matrix given by changing the sign of the elements below the diagonal in M_i^{-1} . Thus,

$$L = M_1 M_2 \cdots M_{n-1}.$$

This gives us the factorization $A = LU$, where L is a unit lower triangular matrix and U is an upper triangular matrix.

3.2 Partial Pivoting

When performing Gaussian elimination, numerical stability can be a concern if the pivot element is too small or even equal to 0. This can lead to large rounding errors in floating-point computations. To mitigate this, partial pivoting is introduced. Partial pivoting involves swapping rows so that the pivot element at each step is the largest, in magnitude, among the available entries in its column. Mathematically, this is equivalent to applying a permutation matrix P , which reorders the rows of A before factorization:

$$PA = LU$$

After pivoting at each step, we can proceed with the Gaussian elimination as described, but now on the permuted matrix PA .

3.3 Solving the system $Ax = b$

Once the matrix A has been factored as $A = LU$ (or $PA = LU$ in the case of partial pivoting), solving the linear system $Ax = b$, can be broken into two simpler steps:

1. Forward substitution: Solve $Ly = b$ (or $Ly = Pb$) for y .
2. Backward substitution: Solve $Ux = y$ for x .

Both steps involve solving triangular systems, which is a computationally efficient process. For forward substitution, $Ly = b$ (or $Ly = Pb$) can be solved by exploiting the fact that L is lower triangular. The process starts by immediately setting y_1 equal to b_1 . Then, y_1 can be substituted into the second equation to solve for y_2 . This algorithm continues recursively until everything is needed to compute y_n . Thus, y can be found with $O(n^2)$ operations. Then y is used in conjunction with U to solve for x . For backward substitution, $Ux = y$ can be solved by exploiting the fact that U is upper triangular. If we refer to the bottom-rightmost element in U as u_{nn} , then x_n can be immediately solved for by computing $x_n = \frac{y_n}{u_{nn}}$. x_n can then be substituted into the second-to-last equation to solve for x_{n-1} . This algorithm continues recursively until everything is needed to compute x_1 . Thus, similarly to y , x can be found with $O(n^2)$ operations.

4 Description of the Algorithm and Implementation

This report discusses and analyzes the routine that computes LU factorization and the routines that compute the triangular solves $Ly = b$ and $Ux = y$.

All routines and their subsequent testers were developed and run in Spyder, an open source python environment included within Anaconda.

4.1 LU Factorization Algorithm

The LU factorization algorithm takes a matrix A as input and updates it in place to store both the lower triangular matrix L and the upper triangular matrix U . This can be done with or without partial pivoting, depending on the method used. The function takes a flag as an input, either 1 or 2. If method 1 is chosen, the algorithm computes the LU factorization with no pivoting, while if method 2 is chosen, the algorithm computes the LU factorization with partial pivoting. In the case of LU factorization without pivoting, the algorithm begins by determining the matrix's dimension n . For each row i , the diagonal element $\alpha = A[i][i]$ is chosen as the pivot. If $\alpha = 0$, the algorithm stops since division by zero would occur, indicating that the matrix cannot be decomposed without pivoting. After identifying the pivot, the algorithm computes the lambda values, which are the elements below the diagonal in column i . These values, $A[k][i] = A[k][i]/\alpha$, represent the ratios used to form L . Once the lambda values are computed, the algorithm updates the elements in the lower right submatrix by subtracting the product of the lambda value and the corresponding elements in the pivot row. In particular, $A[k][j] = A[k][j] - A[k][i] \cdot A[i][j]$. This updates the entries of U . This process continues for each column until the entire matrix has been processed, at which point the algorithm returns A , which contains L in the lower triangular part and U in the upper triangular part.

In the case of LU factorization with partial pivoting, the algorithm introduces an additional step to ensure numerical stability. First, the matrix dimension n is determined, and a permutation vector P is initialized to keep track of any row swaps. Then, for each column i , the algorithm finds the row (`max_row`) that contains the largest magnitude entry in column i . This row is then swapped with the current row if necessary, and the permutation vector P is updated to reflect this change. If no nonzero element is found in the given column or if the largest element is very small in magnitude (below a given threshold), the algorithm will stop and return an error message. As in the no-pivoting method, the algorithm computes the lambda values $A[k][i] = A[k][i]/\alpha$ for the rows below the diagonal. In the same way as the no pivoting method, the elements in the lower right submatrix are updated to reflect the changes based on the new lambda values. After iterating through each column, the algorithm returns the matrix A (containing L and U) and the permutation vector P , which keeps track of the row swaps.

Both versions of the algorithm are computationally efficient, as A is updated to contain L and U , thus bypassing the need to initialize two new matrices. The LU factorization without pivoting has a computational complexity of $O(n^3)$, as the algorithm involves nested loops over the rows and columns of the matrix. However, the no pivoting method is less stable

because it may encounter zero or small pivot elements, leading to division errors or large rounding errors. In contrast, the LU factorization algorithm with partial pivoting is more stable due to the pivot selection and the use of the threshold value, which avoids divisions by small numbers. The overhead of finding the largest pivot element and applying the threshold check slightly increases the computational cost, but it remains $O(n^3)$, with the benefit of improved accuracy. Overall, the algorithm minimizes unnecessary operations by focusing only on the elements below the diagonal for L and within the submatrix for U , making it highly efficient for solving systems of linear equations.

4.2 Triangular Solve Routines

The two algorithms described here solve triangular systems of equations, with one using forward substitution and the other using backward substitution. These methods are used to solve the equations $Lz = v$ and $Uz = v$, respectively, where L is a unit lower triangular matrix and U is an upper triangular matrix. Both algorithms assume that the matrix LU contains both the unit lower triangular matrix L and the upper triangular matrix U , as is typical after performing LU factorization.

In the forward substitution algorithm, `Lv_solver`, the system $Lz = v$ is solved by iterating through the rows of L , which is a unit lower triangular matrix. The vector z is initialized to zeros and will store the solution. The algorithm begins by setting the first element of z directly equal to $v[0]$ because the diagonal element of L is 1 and there are no elements above the diagonal in the first row. After that, for each subsequent element $z[i]$, the algorithm computes the sum of products between the previously solved elements $z[j]$ and the corresponding entries in the unit lower triangular matrix L . This sum is then subtracted from $v[i]$, giving the value of $z[i]$. This process continues for all n rows, and the solution vector z is returned. Since the algorithm only iterates over the lower triangular part of the matrix, the number of operations grows quadratically with the size of the system, specifically $O(n^2)$, which makes it computationally efficient for solving lower triangular systems.

The backward substitution algorithm, `Uv_solver`, is used to solve $Uz = v$, where U is an upper triangular matrix. Similar to the forward substitution method, z is initialized as a zero vector and will store the solution. The algorithm begins by solving for the last element of z , as it is directly determined by the last row of the matrix U , which contains only one non-zero element in the diagonal position. This value is found by dividing $v[n - 1]$ by the diagonal element $U[n - 1][n - 1]$. Then, for each preceding row, the algorithm computes the sum of products between the already solved elements $z[j]$ and the corresponding entries in the upper triangular matrix U . This sum is subtracted from $v[i]$, and the result is divided by the diagonal element $U[i][i]$ to solve for $z[i]$. Like forward substitution, the backward substitution method requires $O(n^2)$ operations because it iterates only through the upper triangular part of the matrix. Its efficiency is comparable to forward substitution, making it well suited for solving upper triangular systems efficiently.

In both algorithms, the triangular structure of the matrix significantly reduces the computational complexity compared to solving a full system of equations. While both dense matrix-vector products and these forward and backward substitution methods require $O(n^2)$

operations, the triangular solves are more computationally efficient. By only working with the nonzero parts of the triangular matrices, the algorithms cut down the number of computations, providing an efficient solution for systems arising from LU factorization.

5 Description of the Experimental Design and Results

5.1 Empirical Tests With Structure: Predictions and Results

5.1.1 Type 1

In this section, we consider a diagonal matrix $A \in \mathbb{R}^{n \times n}$ with positive diagonal elements.

$$A = \begin{bmatrix} \alpha_{11} & 0 & 0 & \dots & 0 \\ 0 & \alpha_{22} & 0 & \dots & 0 \\ 0 & 0 & \alpha_{33} & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & \alpha_{nn} \end{bmatrix}$$

For LU factorization of a diagonal matrix A with no pivoting, the matrix A is already upper triangular, so the U matrix is simply equal to A . The lower triangular matrix L will be the identity matrix I_n , because no elimination steps are required. For partial pivoting, since the diagonal elements of A are all positive and greater than any off-diagonal elements (which are all zero), partial pivoting will have no effect. The resulting factorization will be the same as in the no-pivoting case, with $P_r = I_n$. With complete pivoting, the entire submatrix is searched for the largest element, so P_r and P_c are not necessarily equal to I_n . The diagonal will be reordered so that going down and to the right, the entries become smaller. $L = I_n$ and U will reflect any changes. For all pivoting strategies, the growth factor will be 1, as the magnitude of the entries are not changing at all (only potentially their positions).

In order to test the developed factorization algorithm with matrices of this structure, we passed two matrices (both $n = 5$) through the no pivoting and partial pivoting algorithms. Complete pivoting was not tested.

$$A_1 = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 & 0 \\ 0 & 0 & 3 & 0 & 0 \\ 0 & 0 & 0 & 4 & 0 \\ 0 & 0 & 0 & 0 & 5 \end{bmatrix} \quad A_2 = \begin{bmatrix} 5 & 0 & 0 & 0 & 0 \\ 0 & 4 & 0 & 0 & 0 \\ 0 & 0 & 3 & 0 & 0 \\ 0 & 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

The output of these tests aligned with the theoretical predictions. For A_1 , both the no pivoting and partial pivoting algorithms yielded $L = I_5$, $U = A_1$, and in partial pivoting, $P_r = I_5$. For A_2 , both the no pivoting and partial pivoting algorithms yielded $L = I_5$, $U = A_2$, and in partial pivoting, $P_r = I_5$. For both matrices and for both factoring methods, the growth factor was $\gamma = 1$.

5.1.2 Type 2

In this section, we consider an antidiagonal matrix $A \in \mathbb{R}^{n \times n}$ with positive antidiagonal elements.

$$A = \begin{bmatrix} 0 & 0 & \dots & 0 & \alpha_{1n} \\ 0 & 0 & \dots & \alpha_{2,n-1} & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & \alpha_{n-1,2} & \dots & 0 & 0 \\ \alpha_{n1} & 0 & \dots & 0 & 0 \end{bmatrix}$$

For an antidiagonal matrix, performing LU factorization without pivoting is not possible. Since the initial principle element is always zero, the factoring cannot proceed because a division by zero would be needed. When performing partial pivoting on an antidiagonal matrix, the resulting permutation matrix, P_r , will reorder the rows to turn the antidiagonal matrix into a diagonal matrix. Once this reordering is done, the LU factorization proceeds as it would for a diagonal matrix. In this case, $L = I_n$, U is equal to the diagonal matrix obtained from the reordered A . P_r will be the permutation matrix that reflects the row swaps. With complete pivoting, the matrix will again be transformed into a diagonal matrix with the elements decreasing in magnitude from top-left to bottom-right. $L = I_n$, U is equal to the newly obtained diagonal matrix and P_r and P_c will be permutation matrices that reorder the rows and columns, respectively, to transform the antidiagonal matrix into a diagonal matrix. For all pivoting strategies, the growth factor will be 1, as the magnitude of the entries are not changing at all (only potentially their positions).

In order to test the developed factorization algorithm with matrices of this structure, we passed two matrices (both $n = 5$) through the no pivoting and partial pivoting algorithms. Complete pivoting was not tested.

$$A_1 = \begin{bmatrix} 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 2 & 0 \\ 0 & 0 & 3 & 0 & 0 \\ 0 & 4 & 0 & 0 & 0 \\ 5 & 0 & 0 & 0 & 0 \end{bmatrix} \quad A_2 = \begin{bmatrix} 0 & 0 & 0 & 0 & 5 \\ 0 & 0 & 0 & 4 & 0 \\ 0 & 0 & 3 & 0 & 0 \\ 0 & 2 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \end{bmatrix}$$

The output of these tests aligned with the theoretical predictions. For A_1 , no pivoting did not work and partial pivoting transformed the matrix into A_2 from Type 1. In this case, $L = I_5$, $U = A_2$ (from Type 1), and P_r (in vector form) = $[4, 3, 2, 1, 0]$, thus putting the rows of the matrix in reverse order. For A_2 , no pivoting did not work and partial pivoting transformed the matrix into A_1 from Type 1. In this case, $L = I_5$, $U = A_1$ (from Type 1), and P_r (in vector form) = $[4, 3, 2, 1, 0]$, thus putting the rows of the matrix in reverse order. For both matrices and for both factoring methods, the growth factor was $\gamma = 1$.

5.1.3 Type 3

In this section, we consider a matrix $A \in \mathbb{R}^{n \times n}$ that is the sum of a diagonal matrix and an antidiagonal matrix with positive elements. It is important to note that care must be taken to ensure that one row is not a scalar multiple of another, for this would make the matrix singular and inhibit LU factorization. For the no pivoting case, the structure of the nonzero

pattern may make it challenging to perform factorization without encountering issues. In this case, L will have nonzero elements on the lower triangular part of the antidiagonal and U will likely have fill-ins caused by the interaction between the diagonal and antidiagonal terms. Without pivoting, the factorization could lead to numerical instability due to the small pivots that could appear on the diagonal, especially if the antidiagonal terms are significant relative to the diagonal. For partial pivoting, the algorithm will help to mitigate some numerical instability by ensuring that the pivots are not too small. L will have nonzero entries below the diagonal and U may have nonzero entries resulting from the combination of the diagonal and antidiagonal elements during elimination. P_r will be the permutation matrix reflecting the row swaps performed during the factorization. For complete pivoting, L and U may be similar to the other cases, but the process is more numerically stable. For certain choices of pivot elements, particularly if the diagonal elements are significantly larger than the antidiagonal elements, complete pivoting may tend to select diagonal elements first. In this case, the permutation matrices P_r and P_c could largely preserve the diagonal dominance, and the resulting factorization would yield an L matrix with relatively small entries (close to an identity matrix), and U would remain close to diagonal with only small fill-ins. On the other hand, if the antidiagonal elements dominate, the pivoting process would lead to larger reordering of the matrix, resulting in a more complex L and U . In general, complete pivoting will yield a stable factorization but at the cost of losing much of the original matrix structure.

In order to test the developed factorization algorithm with matrices of this structure, the following 5×5 matrix was passed through the no pivoting and partial pivoting algorithms. Complete pivoting was not tested.

$$A = \begin{bmatrix} 1 & 0 & 0 & 0 & 6 \\ 0 & 2 & 0 & 7 & 0 \\ 0 & 0 & 3 & 0 & 0 \\ 0 & 8 & 0 & 4 & 0 \\ 9 & 0 & 0 & 0 & 5 \end{bmatrix}$$

The output of these tests aligned with the theoretical predictions. No pivoting yielded an L that is somewhat close to the identity, with nonzero entries on the lower triangular part of the antidiagonal, and a U with relatively large magnitude entries on the lower right part of the diagonal due to the updates caused by the interaction between the diagonal and antidiagonal terms. For partial pivoting, the resulting L is the same in form to the no pivoting case, but the nonzero entries are the reciprocals of corresponding entries in the no pivoting L . The resulting U also has the same form as the no pivoting case, but the nonzero entries are slightly different due to the differing Gaussian elimination updates. In this case, $P_r = [4, 3, 2, 1, 0]$, but this is dependent on the choice of pivots in the test matrix.

5.1.4 Type 4

In this section, we consider a matrix $A \in \mathbb{R}^{n \times n}$ that is unit lower triangular and the elements in the strict lower part all have magnitude less than 1. It can be noted that in the case of partial pivoting or complete pivoting, the largest magnitude element, in either the column or entire submatrix, is already the chosen diagonal element, so no swapping occurs in either

method. So in partial pivoting, $P_r = I_n$, and in complete pivoting, $P_r = P_c = I_n$. For all three factorization methods, Gaussian elimination will occur to eliminate the elements below the diagonal, but A is already unit lower triangular, so the elements placed in L will be identical to those in the corresponding positions of A . Consequently, U will always be the identity matrix. So $L = A$ and $U = I_n$.

In order to test the developed factorization algorithm with matrices of this structure, the following 5×5 matrix was passed through the no pivoting and partial pivoting algorithms. Complete pivoting was not tested.

$$A = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0.1 & 1 & 0 & 0 & 0 \\ -0.2 & 0.5 & 1 & 0 & 0 \\ 0.7 & -0.6 & 0.4 & 1 & 0 \\ 0.8 & -0.9 & 0.3 & -0.2 & 1 \end{bmatrix}$$

The output of these tests aligned with the theoretical predictions. Both tested factoring methods produced $L = A$ and $U = I_5$. No swapping took place with partial pivoting, so $P_r = I_5$.

5.1.5 Type 5

In this section, we consider a lower triangular matrix $A \in \mathbb{R}^{n \times n}$ where the diagonal elements are positive and not 1 and the elements in the strictly lower triangular part are larger than 1. It should be noted that if all the elements in the lower triangular part are strictly less than the elements on the diagonal, the results will be very similar to those outlined in Type 4. No pivoting would be necessary (other than possible reordering of the diagonal with complete pivoting) and L and U would be easily surmised from A . Consider the case where the entries below the diagonal are larger than the entries in the diagonal. For the no pivoting case, L will contain values from the strictly lower part of A , normalized to have ones on the diagonal, i.e., the elements in L will be the ratios of the elements in A to their corresponding diagonal entries. U will simply be a diagonal matrix with the diagonal entries of A . In the case of partial pivoting, the rows will be swapped if an element larger in magnitude than the pivot is found. L and U then would not necessarily follow the same nice structure as the no pivoting case, but may contain several nonzero off diagonal entries. With complete pivoting, both rows and columns may be swapped to improve stability, but L and U will again not necessarily have nice structure.

In order to test the developed factorization algorithm with matrices of this structure, the following 5×5 matrix was passed through the no pivoting and partial pivoting algorithms. Complete pivoting was not tested.

$$A = \begin{bmatrix} 2 & 0 & 0 & 0 & 0 \\ 3 & 2 & 0 & 0 & 0 \\ 4 & 3 & 2 & 0 & 0 \\ 5 & 4 & 3 & 2 & 0 \\ 6 & 5 & 4 & 3 & 2 \end{bmatrix}$$

The output of these tests aligned with the theoretical predictions. In the no pivoting case, the values in L are ratios of the corresponding elements in A to the diagonal entries in the same row. For example, the entry in the second row and first column of L is $\frac{3}{2} = 1.5$. It was also found that, as expected, U is a diagonal matrix with 2's on the diagonal. With partial pivoting, P_r was found to be $[4, 0, 1, 2, 3]$ and L and U do not have nice structure as they do in the no pivoting case.

5.1.6 Type 6

In this section, we consider a lower triangular matrix $A \in \mathbb{R}^{n \times n}$ that is tridiagonal with all elements on the main diagonal and the first super and subdiagonals nonzero. Additionally, A is strictly diagonally dominant by rows and columns. Because the tridiagonal structure is preserved in the factorization, the no pivoting algorithm would yield L with one nonzero subdiagonal and U with one nonzero superdiagonal. With partial pivoting, no row swaps are necessary since the matrix is diagonally dominant. L and U will maintain their bidiagonal structure and $P_r = I_n$. With complete pivoting, an entry further down the diagonal may be swapped with a relatively smaller entry, which may disrupt the tridiagonal structure. In this case, P_r and P_c would track these swaps, but we do not have necessarily nice structure for L and U .

In order to test the developed factorization algorithm with matrices of this structure, the following 5×5 matrix was passed through the no pivoting and partial pivoting algorithms. Complete pivoting was not tested.

$$A = \begin{bmatrix} 14 & 4 & 0 & 0 & 0 \\ 3 & 17 & 7 & 0 & 0 \\ 0 & 6 & 12 & 1 & 0 \\ 0 & 0 & 4 & 18 & 5 \\ 0 & 0 & 0 & 8 & 13 \end{bmatrix}$$

The output of these tests aligned with the theoretical predictions. The result of the no pivoting algorithm shows that L is banded with one subdiagonal and U is banded with one superdiagonal. The partial pivoting output shows this same structure within L and U , and we have $P_r = I_5$, as expected.

5.1.7 Type 7

In this section, we consider a matrix $A \in \mathbb{R}^{n \times n}$ where all elements below the diagonal are -1 , all elements on the diagonal are 1, and all elements in the last column are 1, with all other elements being 0.

$$A = \begin{bmatrix} 1 & 0 & 0 & \dots & 1 \\ -1 & 1 & 0 & \dots & 1 \\ -1 & -1 & 1 & \dots & 1 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ -1 & -1 & -1 & \dots & 1 \end{bmatrix}$$

For LU factorization with no pivoting, the 1's on the diagonal will be used to eliminate the -1 's in each column, however, the multipliers needed to eliminate these -1 's are all -1 , so

L will be unit lower triangular with every entry below the diagonal being equal to -1 . In the process of eliminating these -1 's, the entries in the last column of the submatrix are updated. More specifically, at each step, the entries in the last column beneath the current pivot row will be doubled. Thus, U will contain 1's on the diagonal, and zeros everywhere else apart from the last column, where the entry in the i th row will be 2^{i-1} . In the case of partial pivoting, L and U should stay exactly the same, as the largest magnitude entry in every column will be the 1 on the diagonal, so no row swaps are needed. Thus, $P_r = I_n$. With complete pivoting, the algorithm is not as smooth. After the first step, when the last column is populated with 2's apart from the first entry, one of these 2's will be swapped to the current pivot position. This messes with the structure and L and U will likely not have clear structure anymore. P_r should be equal to I_n since the largest magnitude element can always be chosen as the entry in the last column of the same row. Thus, P_c would represent the swaps needed to move the entries from the last column to the current pivot column. The growth factor for no pivoting and partial pivoting should be the same, and with all three methods, we are expected to have $\gamma > 1$, since the magnitudes of the elements in the last column are increasing.

In order to test the developed factorization algorithm with matrices of this structure, the following 5×5 matrix was passed through the no pivoting and partial pivoting algorithms. Complete pivoting was not tested.

$$A = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 \\ -1 & 1 & 0 & 0 & 1 \\ -1 & -1 & 1 & 0 & 1 \\ -1 & -1 & -1 & 1 & 1 \\ -1 & -1 & -1 & -1 & 1 \end{bmatrix}$$

The output of these tests aligned with the theoretical predictions. The no pivoting and partial pivoting methods produce the same L and U where L is I_5 with every entry below the diagonal equal to -1 and U is I_5 with the entries in the last column being 1, 2, 4, 8, 16 as one goes down. Additionally, $P_r = I_5$. The growth factor is also the same in both methods: $\gamma \approx 8.14$.

5.1.8 Type 8

In this section, we consider a matrix $A \in \mathbb{R}^{n \times n}$ where A is a symmetric positive definite matrix generated from a lower triangular $\tilde{L}$ with positive diagonal elements (not necessarily 1) via $A = \tilde{L}\tilde{L}^T$. Since A is generated in this way, we know it is nonsingular and the factorization will succeed without pivoting. Thus, partial and complete pivoting are not considered here. Notably, there is a relationship between the $\tilde{L}$ used to generate A and the L and U that result from the factorization. L is simply $\tilde{L}$ normalized to have 1's on the diagonal. Thus each element in L will be the ratio of the corresponding element in $\tilde{L}$ to the diagonal element in that column of $\tilde{L}$. Each row of U is a scalar multiple of the corresponding row of $\tilde{L}^T$, with the scalar being the diagonal element in that row.

In order to test the developed factorization algorithm with matrices of this structure, the

following $\tilde{L}$ and its accompanying $\tilde{L}^T$ were used to generate the following $A = \tilde{L}\tilde{L}^T$.

$$\tilde{L} = \begin{bmatrix} 3 & 0 & 0 & 0 & 0 \\ 5 & 8 & 0 & 0 & 0 \\ 7 & 3 & 6 & 0 & 0 \\ 4 & 9 & 1 & 2 & 0 \\ 1 & 4 & 7 & 6 & 9 \end{bmatrix} \quad \tilde{L}^T = \begin{bmatrix} 3 & 5 & 7 & 4 & 1 \\ 0 & 8 & 3 & 9 & 4 \\ 0 & 0 & 6 & 1 & 7 \\ 0 & 0 & 0 & 2 & 6 \\ 0 & 0 & 0 & 0 & 9 \end{bmatrix} \quad A = \tilde{L}\tilde{L}^T = \begin{bmatrix} 9 & 15 & 21 & 12 & 3 \\ 15 & 89 & 59 & 92 & 37 \\ 21 & 59 & 94 & 61 & 61 \\ 12 & 92 & 61 & 102 & 59 \\ 3 & 37 & 61 & 59 & 183 \end{bmatrix}$$

After passing A through the no pivoting algorithm, the following L and U were obtained.

$$L = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 5/3 & 1 & 0 & 0 & 0 \\ 7/3 & 3/8 & 1 & 0 & 0 \\ 4/3 & 9/8 & 1/6 & 1 & 0 \\ 1/3 & 1/2 & 7/6 & 3 & 1 \end{bmatrix} \quad U = \begin{bmatrix} 9 & 15 & 21 & 12 & 3 \\ 0 & 64 & 24 & 72 & 32 \\ 0 & 0 & 36 & 6 & 42 \\ 0 & 0 & 0 & 4 & 12 \\ 0 & 0 & 0 & 0 & 81 \end{bmatrix}$$

The output of these tests aligned with the theoretical predictions. It can be seen that each element in L is the ratio of the corresponding element in $\tilde{L}$ to the diagonal element in that column of $\tilde{L}$. Additionally, each row of U is a scalar multiple of the corresponding row of $\tilde{L}^T$, with the scalar being the diagonal element in that row.

5.2 Generation of Random Test Problems

Special care was taken in regards to how the random test problems were generated. In order for the LU factorization routine to be successful, the random A 's had to be nonsingular. Since each A was computed as the product of a random L and U , L and U were generated to each be nonsingular, so that their product would be as well. Since L is unit lower triangular, it is nonsingular by definition. In the generation of U , the entries on the diagonal were restricted from being 0 in order to ensure its nonsingularity. To generate the matrices, the user inputs a positive number that serves as the magnitude upper bound for the entries. For example, if the user inputs 30 as the upper bound, the entries will be floats between -30 and 30 . Additional precautions were taken to guarantee that the condition number of A , computed as $\|A\|_2 \cdot \|A^{-1}\|_2$, did not get too large. High condition number signifies that a matrix is close to singular and thus may become unstable during the factorization. In order to reduce the condition number of the generated A , the following steps were taken:

1. The entries below the diagonal in L were set to be < 1 in magnitude. This ensures that L does not introduce large values that would amplify the condition number when multiplying with U .
2. The entries on the diagonal of U are set to be > 1 in magnitude. For example, if the user inputs 30 as the upper bound, entries on the diagonal of U will be in $[-30, -1] \cup [1, 30]$. This makes sure that one element of the diagonal is not very large relative to another element of the diagonal.
3. 3 times the user inputted upper bound times the identity matrix is added to the generated U . For example, if the user inputs 30 as the upper bound, $90 \cdot I$ will be added to U . This ensures the diagonal dominance of the matrix, a key factor of nicely conditioned problems.

As a result of these precautions, the random test matrices are nonsingular, well-conditioned, and ready to be passed through the LU factorization routine.

5.3 Description of the Random Tester

The tester first asks the user to specify which method of factorization they would like to test: no pivoting or partial pivoting. Note that the methods are tested separately to ensure that the random problems passed through the partial pivoting algorithm actually require some pivoting. If the same test problems were passed through both algorithms, we would either have a case where many of the problems passed through the partial pivoting routine do not actually require pivoting or all the matrices require pivoting and the no pivoting routine cannot be tested. The tester then asks the user for a lower and upper bound for dimension, number of trials per dimension, and an upper bound for the magnitude of the float entries in L and U . The tester then generates the appropriate random matrices, as outlined in section 5.2. For example, suppose the user inputs the bounds for dimension as 50 and 100, inputs the number of trials per dimension as 10, and inputs the magnitude upper bound as 30. Then the tester will generate 10 50×50 matrices, 10 51×51 matrices, 10 52×52 matrices, all the way up to 10 100×100 matrices. Every matrix will be the product of an L and U with entries less than 30 in magnitude. Additionally, every matrix A is generated along with a random vector x of the appropriate dimension, to be used for the testing of the solver routines. x is populated with floats with the same specified upper bound for magnitude. All random matrices and vectors are generated using the numpy python package.

In order to test the no pivoting algorithm, the tester does the following for each matrix-vector pairing (A and x):

1. Creates a copy of A to be used for later error calculations.
2. Factors A into L and U , where L is stored in the lower triangular part of the matrix and U is stored in the upper triangular part of the matrix. The updated matrix containing L and U is called B .
3. Computes $C = L \cdot U$ using the L and U stored in B .
4. Computes the factorization error: $\frac{||A - C||}{||A||}$ using the Frobenius norm.
5. Computes $Ax = b$ to find b (using the copy of the original A).
6. Solves $Ly = b$ for y , where L is the lower triangular part of B .
7. Solves $U\tilde{x} = y$ for $\tilde{x}$, where U is the upper triangular part of B .
8. Computes the solver error: $\frac{||x - \tilde{x}||}{||x||}$ using the Euclidean norm, where x is the true generated x and $\tilde{x}$ is the computed x .
9. Computes the residual: $\frac{||b - A\tilde{x}||}{||b||}$ using the Euclidean norm.

10. Computes the growth factor: $\gamma = \frac{\| |L||U| \|}{\|A\|}$ using the Frobenius norm, where L and U are the computed factors of A .

In order to test the partial pivoting algorithm, the tester does the following for each matrix-vector pairing (A and x):

1. Generates a random permutation for an $n \times n$ matrix.
2. Applies the random permutation to A , thus creating a new A , which will be our test problem. The original A must be permuted because it was created to be diagonally dominant, so if tested as is, no row swaps would be necessary. (From now on, A will refer to the new, permuted test problem).
3. Creates a copy of A to be used for later error calculations.
4. Factors A into L and U , where L is stored in the lower triangular part of the matrix and U is stored in the upper triangular part of the matrix. The updated matrix containing L and U is called B . The permutation matrix is stored in P , a vector of length n that accounts for the row swaps.
5. Computes $C = L \cdot U$ using the L and U stored in B .
6. Applies the permutation P to the copy of A to find PA .
7. Computes the factorization error: $\frac{\|PA - C\|}{\|A\|}$ using the Frobenius norm.
8. Computes $Ax = b$ to find b (using the copy of the original A).
9. Applies the permutation P to b to find $Pb = \tilde{b}$.
10. Solves $Ly = \tilde{b}$ for y , where L is the lower triangular part of B .
11. Solves $U\tilde{x} = y$ for $\tilde{x}$, where U is the upper triangular part of B .
12. Computes the solver error: $\frac{\|x - \tilde{x}\|}{\|x\|}$ using the Euclidean norm, where x is the true generated x and $\tilde{x}$ is the computed x .
13. Computes the residual: $\frac{\|\tilde{b} - PA\tilde{x}\|}{\|\tilde{b}\|}$ using the Euclidean norm.
14. Computes the growth factor: $\gamma = \frac{\| |L||U| \|}{\|A\|}$ using the Frobenius norm, where L and U are the computed factors of A .

Irrespective of the method used (no pivoting or partial pivoting), the tester then calculates the mean factorization error, mean solver error, mean residual, and mean growth factor for the trials at each dimension. Finally, for each of these statistics, the mean for each dimension is displayed in a scatter plot with the dimension n as the independent variable.

5.4 Results

For both methods of factorization (no pivoting and partial pivoting), two experiments are presented in this report:

1. 10 $n \times n$ matrices are passed through the tester for $n \in \{10, \dots, 100\}$. All entries in the generators L and U were floats with magnitude less than 50. All entries in x were floats with magnitude less than 50.
2. 5 $n \times n$ matrices are passed through the tester for $n \in \{170, \dots, 200\}$. All entries in the generators L and U were floats with magnitude less than 30. All entries in x were floats with magnitude less than 30.

These experiments were constructed to demonstrate the accuracy of the code. A high number of medium-sized matrices and a limited number of larger matrices are passed through the routines to demonstrate that the code works for varying levels of dimension.

Figure 1 shows the results of Experiment 1 with no pivoting. The mean factorization error increases as the dimension increases, in what looks like a linear fashion. However, these errors are all of order 10^{-15} , indicating that any error seen here can likely be attributed to machine and roundoff error. The mean solver error seems to be virtually zero initially, but increases exponentially as the dimension of the test increases. Still, however, these errors stay below order 10^{-10} , indicating that this error can again likely be attributed to machine and roundoff error. The mean residual also increases as the dimension increases. The relationship appears to be linear, but the correlation is weaker than as seen with factorization error. Again, these errors are incredibly small, at order 10^{-16} , meaning it is likely down to machine and roundoff error during the algorithm. The growth factor looks to increase logarithmically, starting at about 1.4 for 10×10 tests and increasing to about 4.8 for 100×100 tests. This makes sense because as the test matrices become larger, the algorithm will increase the magnitude of the elements more and more.

Figure 2 shows the results of Experiment 2 with no pivoting. The mean factorization error looks very similar to Experiment 1, linearly increasing with dimension. The errors are all of order 10^{-15} , indicating that this is likely machine error. The mean solver error again seems to potentially increase exponentially, this time reaching as high as 0.006. This is still quite small and could possibly be attributed to roundoff, but it is also possible that for the larger matrices, the condition number is getting quite high and causing problems for the solver routine. At the higher dimensions, the mean residual seems to lose the linear relationship, but this could be attributed to the relatively small number of tests. The residual is still consistently order 10^{-16} , meaning that it is likely down to machine and roundoff error. The growth factor appears to increase linearly (could likely be logarithmically if we zoom out) from about 6.4 for 170×170 tests to about 7.1 for 200×200 tests. The higher growth number makes sense for the larger matrices, as there is more room for large magnitude increases.

Figure 3 shows the results of Experiment 1 with partial pivoting. The mean factorization error increases as the dimension increases, in what looks like this time a logarithmic fashion. However, these errors are all of order 10^{-16} , indicating that any error seen here can likely be attributed to machine and roundoff error. The mean solver error seems to be virtually zero

initially, but increases exponentially as the dimension of the test increases. Still, however, these errors stay below order 10^{-9} , indicating that this error can again likely be attributed to machine and roundoff error. The mean residual also increases as the dimension increases. The relationship appears to be linear (or possibly logarithmic) with a semi-strong correlation. Again, these errors are incredibly small, at order 10^{-16} , meaning it is likely down to machine and roundoff error during the algorithm. The growth factor looks to increase logarithmically, starting at about 1.4 for 10×10 tests and increasing to about 4.8 for 100×100 tests. This makes sense because as the test matrices become larger, the algorithm will increase the magnitude of the elements more and more. This also shows that pivoting does not seem to have a large impact on growth factor as the plots for no pivoting and partial pivoting look very similar.

Figure 4 shows the results of Experiment 2 with partial pivoting. The mean factorization error looks very similar to Experiment 1, linearly increasing with dimension. The errors are all of order 10^{-16} , indicating that this is likely machine error. The mean solver error again seems to potentially increase exponentially, this time reaching as high as 0.004. This is still quite small and could possibly be attributed to roundoff, but it is also possible that for the larger matrices, the condition number is getting quite high and causing problems for the solver routine, similar to the no pivoting case. At the higher dimensions, the mean residual seems to lose the linear relationship, but this could be attributed to the relatively small number of tests. The residual is still consistently order 10^{-16} , meaning that it is likely down to machine and roundoff error. The growth factor appears to increase linearly (could likely be logarithmically if we zoom out) from about 6.4 for 170×170 tests to about 7.1 for 200×200 tests. The higher growth number makes sense for the larger matrices, as there is more room for large magnitude increases. This also shows that even at these high dimensions, pivoting does not seem to have a large impact on growth factor as the plots for no pivoting and partial pivoting look very similar.

Throughout all the experiments, very little factorization error was observed, highlighting the accuracy of the factorization algorithms. Additionally, very little error was seen in the solver, although it did increase as the dimension increased. This could be attributed to a higher condition number for the large matrices that indicates their slight numerical instability. The residual error seen was likely all down to machine and roundoff error and the growth factor calculations seemed to match with expectations. Overall, this rigorous testing process showed that the algorithms work well.

6 Conclusions

LU factorization is a fundamental tool in numerical linear algebra, widely used for solving systems of linear equations of the form $Ax = b$. By decomposing matrix A into the product of a unit lower triangular matrix L and an upper triangular matrix U , LU factorization simplifies the process of solving such systems. Once the factorization is complete, the problem can be solved efficiently using forward and backward substitution, which is computationally less expensive than directly solving the system. This approach also avoids the need for matrix inversion, which can be numerically unstable for large or ill-conditioned matrices.

This report has demonstrated the accuracy and robustness of the LU factorization routines, both with and without partial pivoting, through a series of carefully designed experiments. The results align with theoretical predictions, indicating that the implemented algorithms perform as expected. The data presented show no major outliers or unexpected anomalies, indicating that the code behaves as expected across different matrix sizes. Minor increases in solver error and growth factor for larger matrices suggest that numerical stability may be strained at higher dimensions, but this is more likely an understood limitation of floating-point arithmetic rather than a bug in the code. Overall, the results of this report strongly suggest that the LU factorization and solving routines are correctly implemented and highly accurate. LU factorization, as a method for efficiently solving systems of equations, remains a cornerstone of computational mathematics, and the routines implemented in this project are shown to perform that task effectively.

7 Figures

7.1 No Pivoting

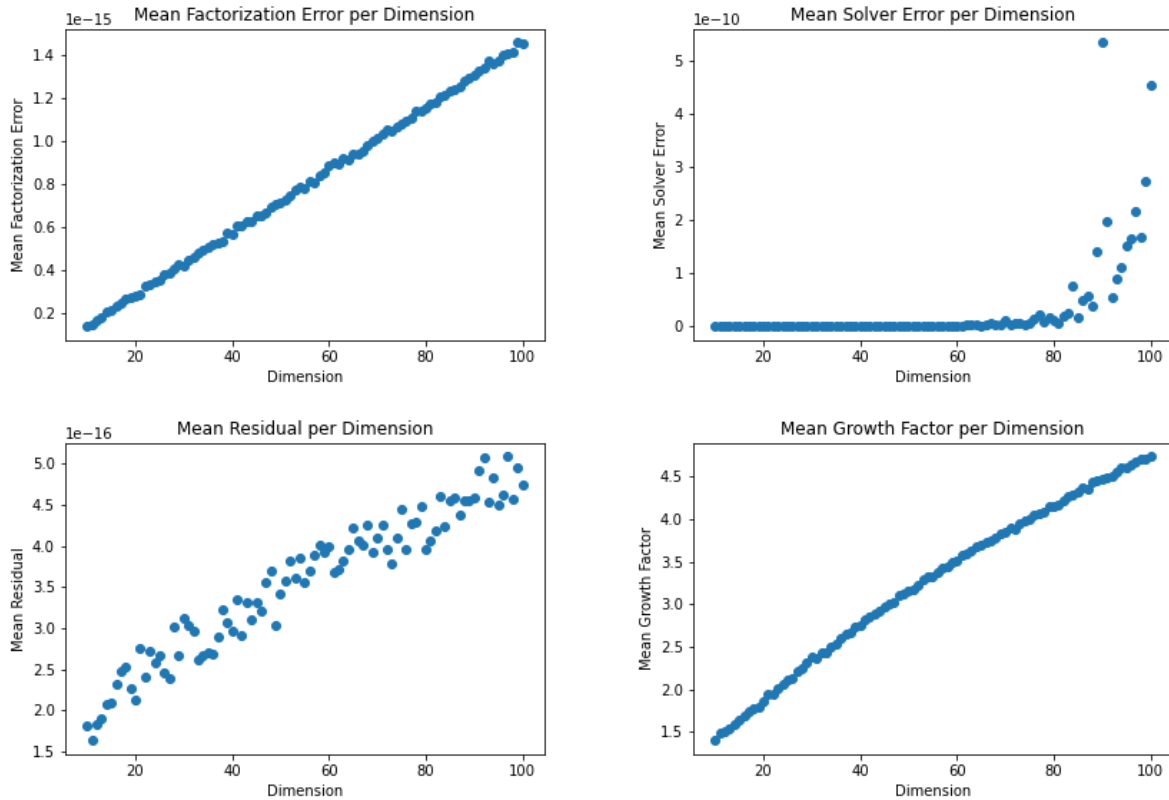


Figure 1. Results for Experiment 1 with no pivoting. 10 trials per dimension for $10 \leq n \leq 100$. All entries in the generators L and U were floats with magnitude less than 50. All entries in x were floats with magnitude less than 50.

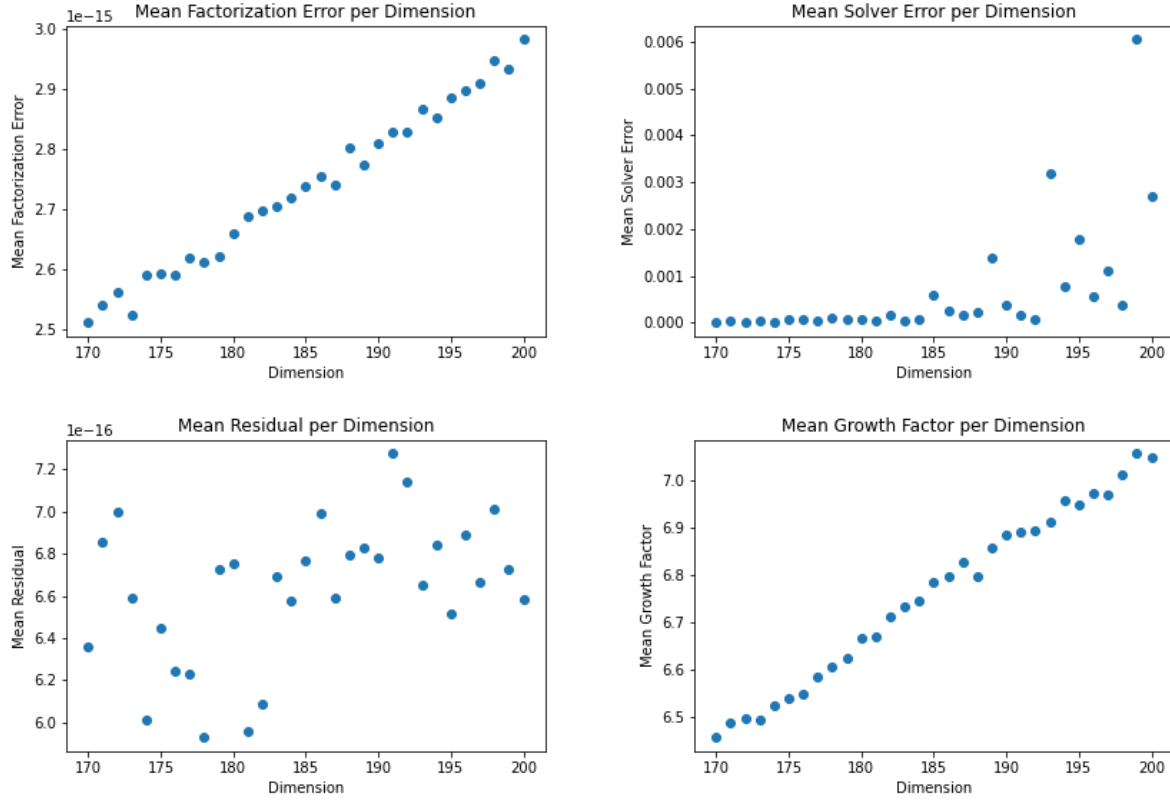


Figure 2. Results for Experiment 2 with no pivoting. 5 trials per dimension for $170 \leq n \leq 200$. All entries in the generators L and U were floats with magnitude less than 30. All entries in x were floats with magnitude less than 30.

7.2 Partial Pivoting

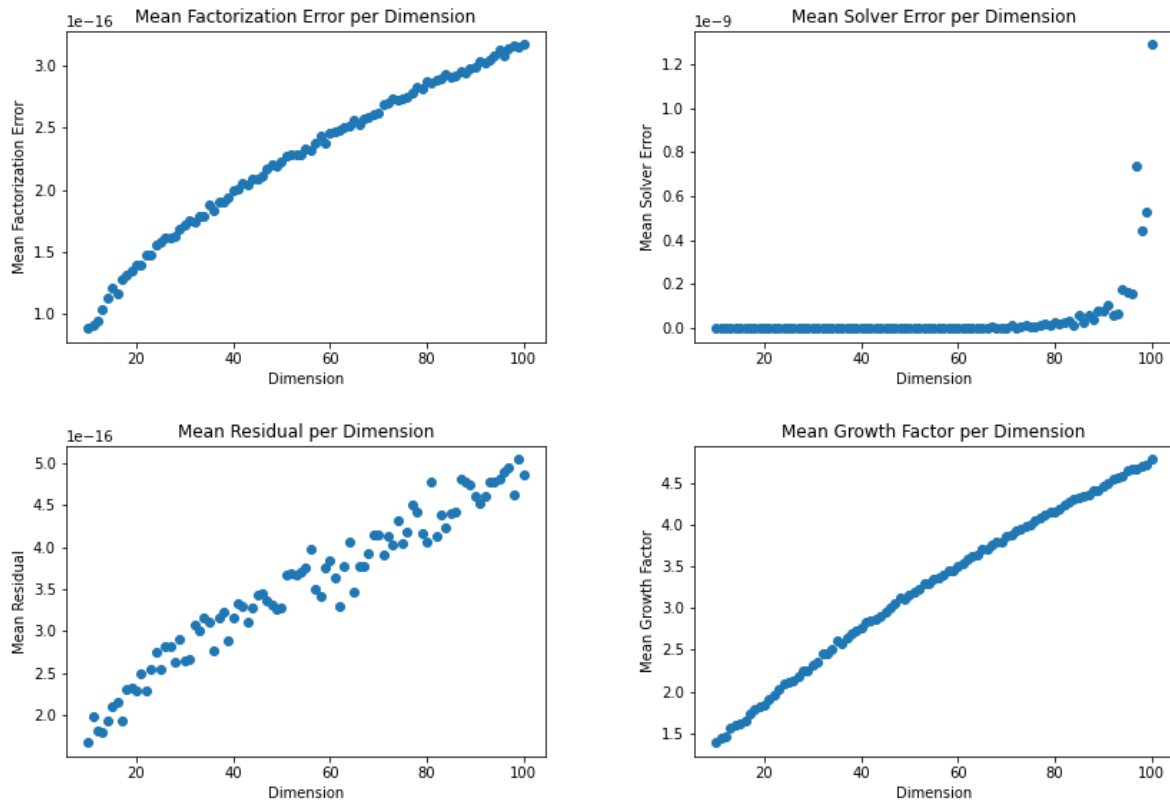


Figure 3. Results for Experiment 1 with partial pivoting. 10 trials per dimension for $10 \leq n \leq 100$. All entries in the generators L and U were floats with magnitude less than 50. All entries in x were floats with magnitude less than 50.

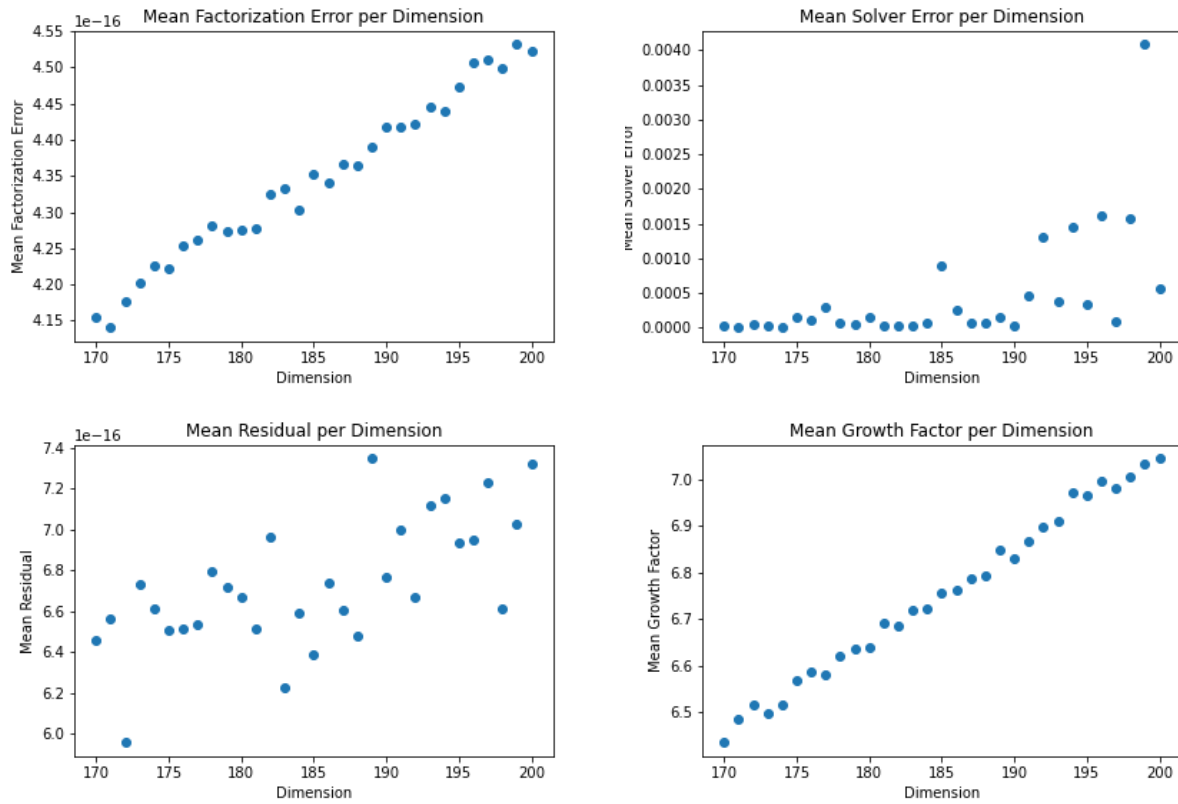


Figure 4. Results for Experiment 2 with partial pivoting. 5 trials per dimension for $170 \leq n \leq 200$. All entries in the generators L and U were floats with magnitude less than 30. All entries in x were floats with magnitude less than 30.

8 References

In the creation of this report, I referenced "Set 4: Factorization for Linear Systems Part 2". I also consulted with Dan Mazus about algorithm development. We wrote out some pseudocode together.